AAI3008
Large Language Models Review

Zhang Zhengchen
zhengchen.zhang@singaporetech.edu.sg

Module Roadmap

Data & Representation

- Tokenize
- Embedding

Model Architecture

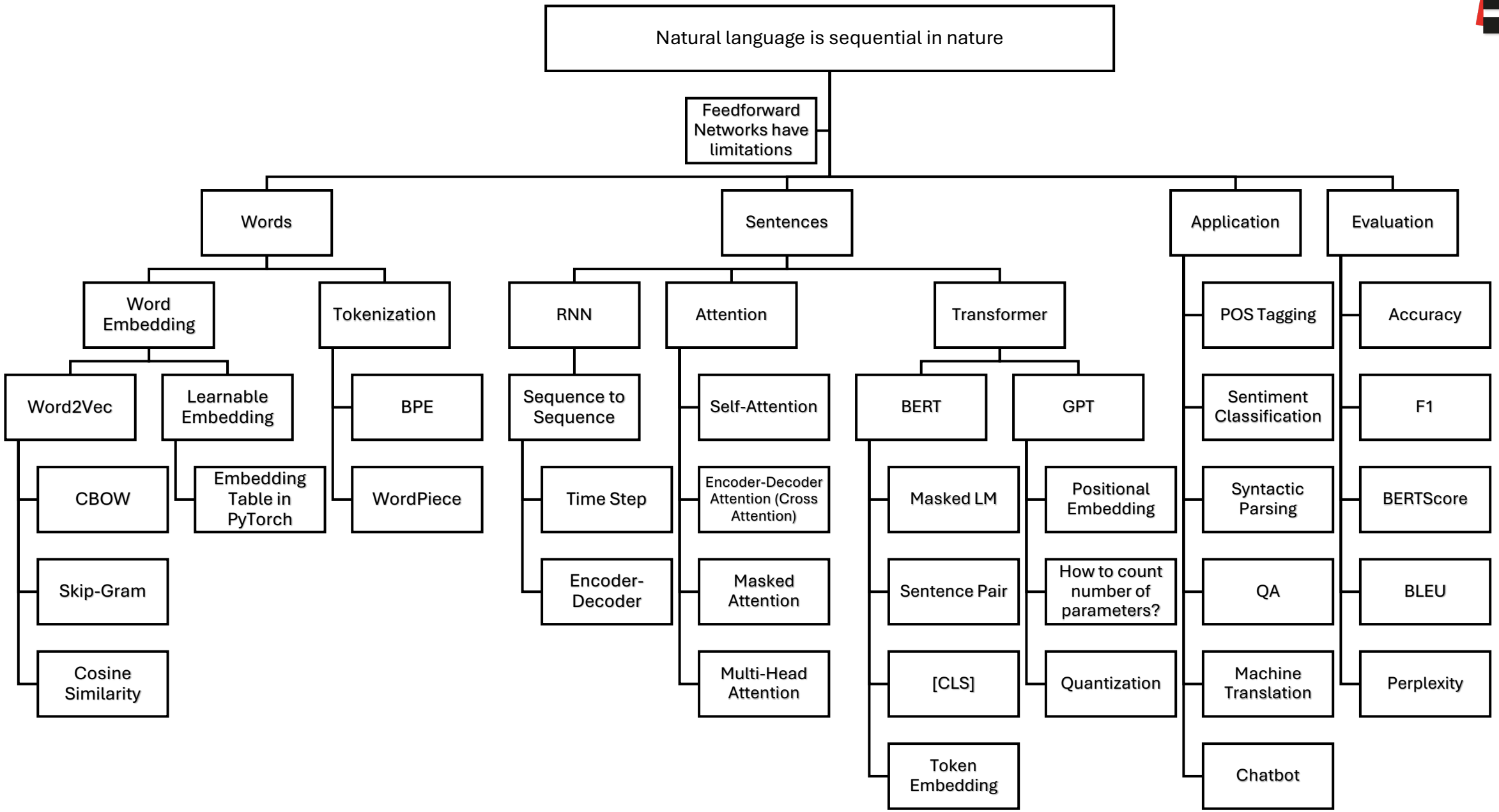
- Transformer
- Recurrent Neural Network
- LLM Adaptation

Tasks & Evaluation

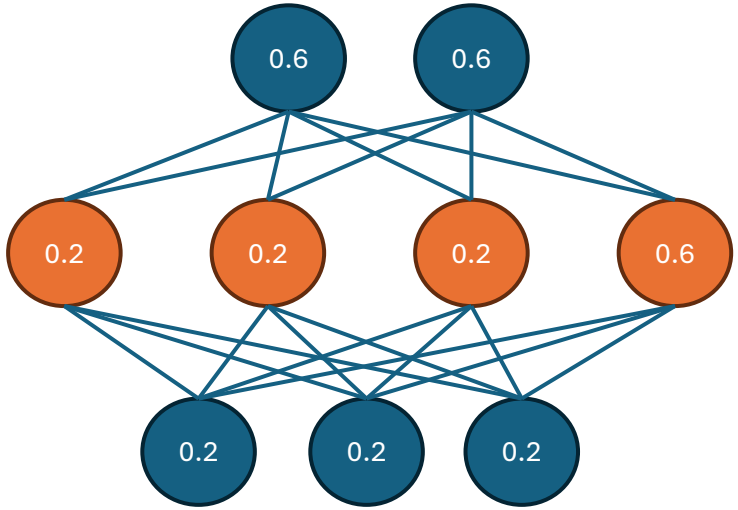
- Question & Answering
- Machine Translation
- Chatbot

System & Applications

- Prompt Engineering
- Retrieval Augmented Generation (RAG)
- Spoken Language Processing
- Multimodal LLM


Natural language is sequential in nature
Feedforward Networks have limitations
Words
Sentences
Application
Evaluation
Word Embedding
Tokenization
RNN
Attention
Transformer
POS Tagging
Accuracy
Word2Vec
Learnable Embedding
BPE
Sequence to Sequence
Self-Attention
BERT
GPT
Sentiment Classification
F1
CBOW
Embedding Table in PyTorch
WordPiece
Time Step
Encoder-Decoder Attention (Cross Attention)
Masked LM
Positional Embedding
Syntactic Parsing
BERTScore
Skip-Gram
Encoder-Decoder
Masked Attention
Sentence Pair
How to count number of parameters?
QA
BLEU
Cosine Similarity
Multi-Head Attention
[CLS]
Quantization
Machine Translation
Perplexity
Token Embedding
Chatbot

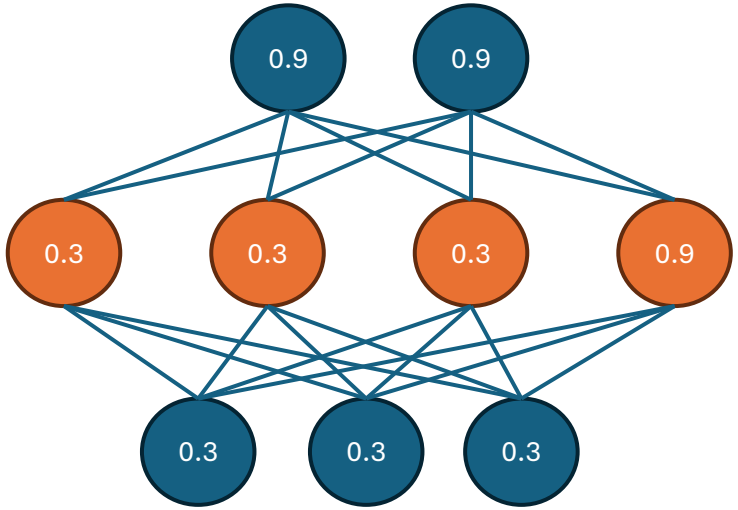
FNN to RNN



$$x = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.2 \end{bmatrix} \qquad W_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

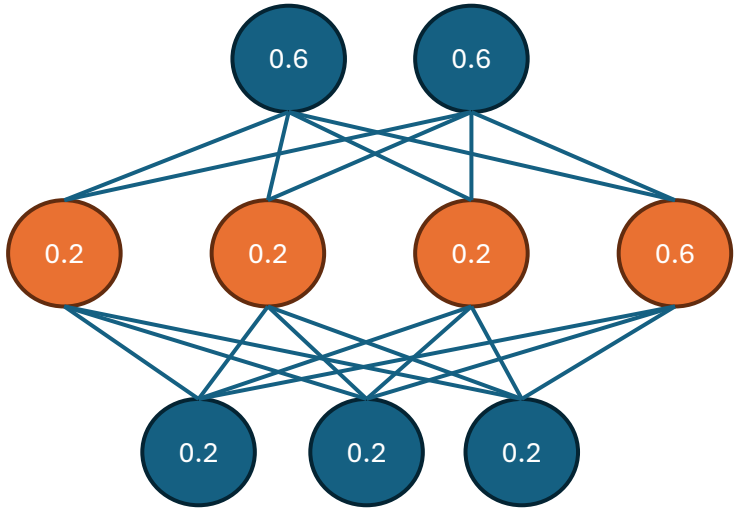
$$h = W_x x$$

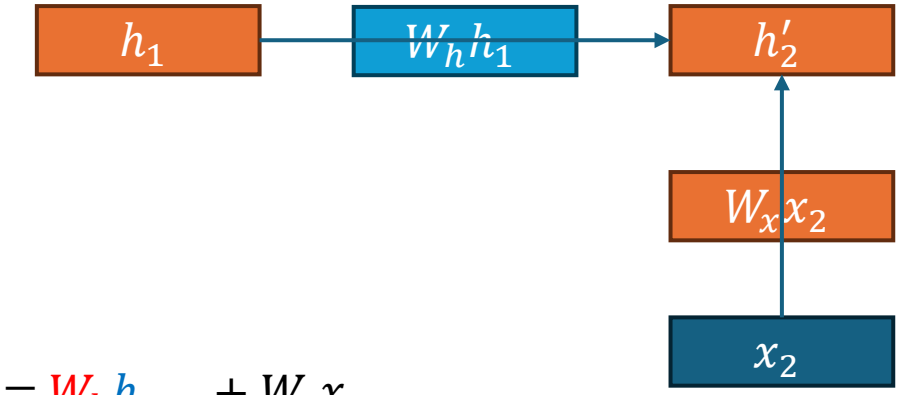
$$h_1 = [0.2 \quad 0.2 \quad 0.2 \quad 0.6]$$



$$x = \begin{bmatrix} 0.3 \\ 0.3 \\ 0.3 \end{bmatrix} \qquad W_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h_2 = [0.3 \quad 0.3 \quad 0.3 \quad 0.9]$$





$$x = \begin{bmatrix} 0.2 \\ 0.2 \\ 0.2 \end{bmatrix} \qquad W_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

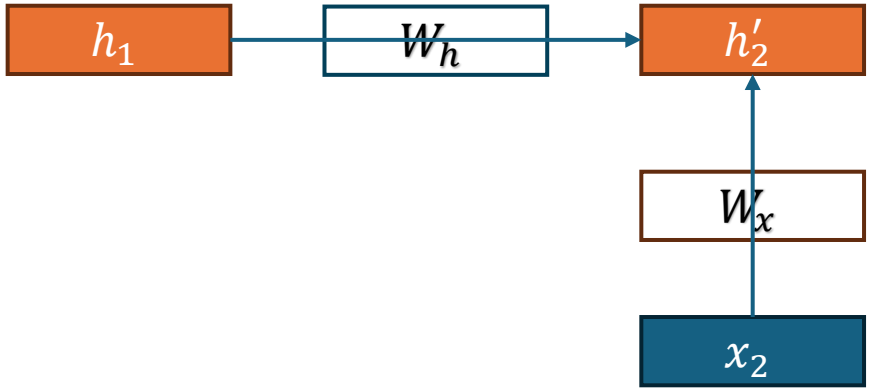
$$h = W_x x$$

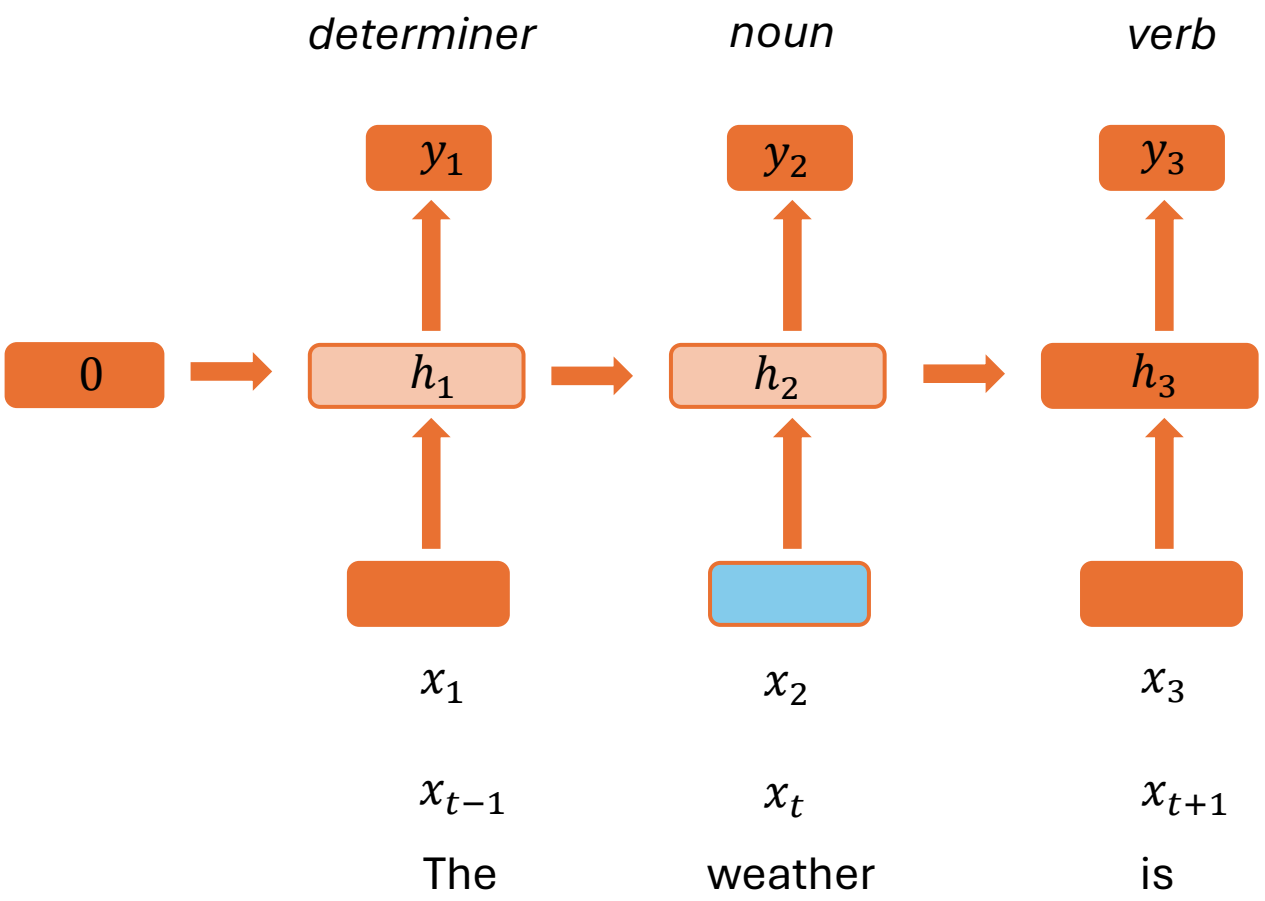
$$h_1 = [0.2 \quad 0.2 \quad 0.2 \quad 0.6]^T \qquad h_2 = [0.3 \quad 0.3 \quad 0.3 \quad 0.9]^T$$

$$h_t = W_h h_{t-1} + W_x x_t$$

$$\mathrm{W}_h = I$$

$$h_2' = I * h_1 + h_2 = [0.5 \quad 0.5 \quad 0.5 \quad 1.5]^T$$





$$h_t = W_h h_{t-1} + W_x x_t$$

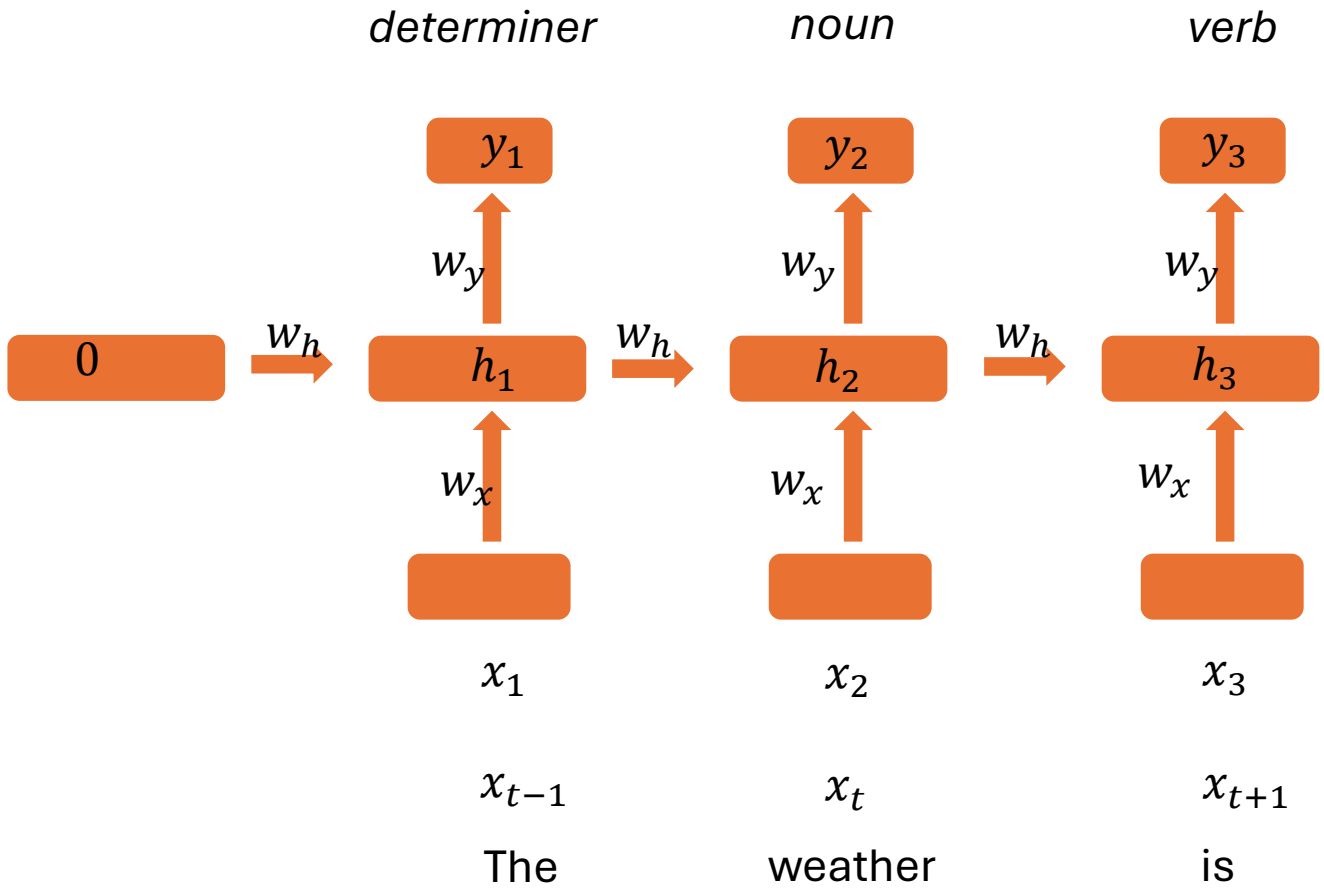
$$W_h = I$$

$$h_2' = I * h_1 + h_2 = [0.5 \quad 0.5 \quad 0.5 \quad 1.5]^T$$

$$h_t = f(W_h h_{t-1} + W_x x_t + bias)$$

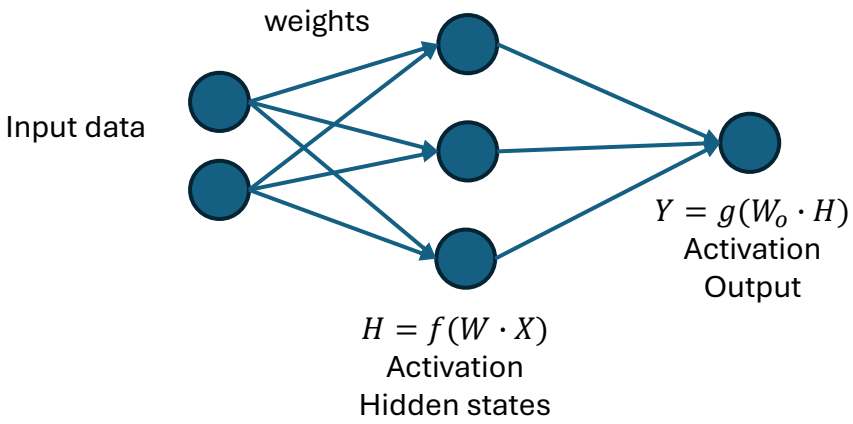
RNN

- $h_t = f(W_h h_{t-1} + W_x x_t + b)$
- $y_t = g(W_y h_t + b_y)$
  - $\mathrm{Y} = \mathrm{Softmax}(W_2 h + b_2)$
- Time Step t
  - A word
  - A segment of audio
  - Stock price of a day



Feedforward Network

- $h = \mathrm{ReLU}(W_1 \mathrm{X} + b_1)$
- $\mathrm{Y} = \mathrm{Softmax}(W_2 h + b_2)$
- $\mathrm{Loss} = \frac{1}{n}\sum_{i=1}^{n}(\widehat{y_i} - y_i)^2$



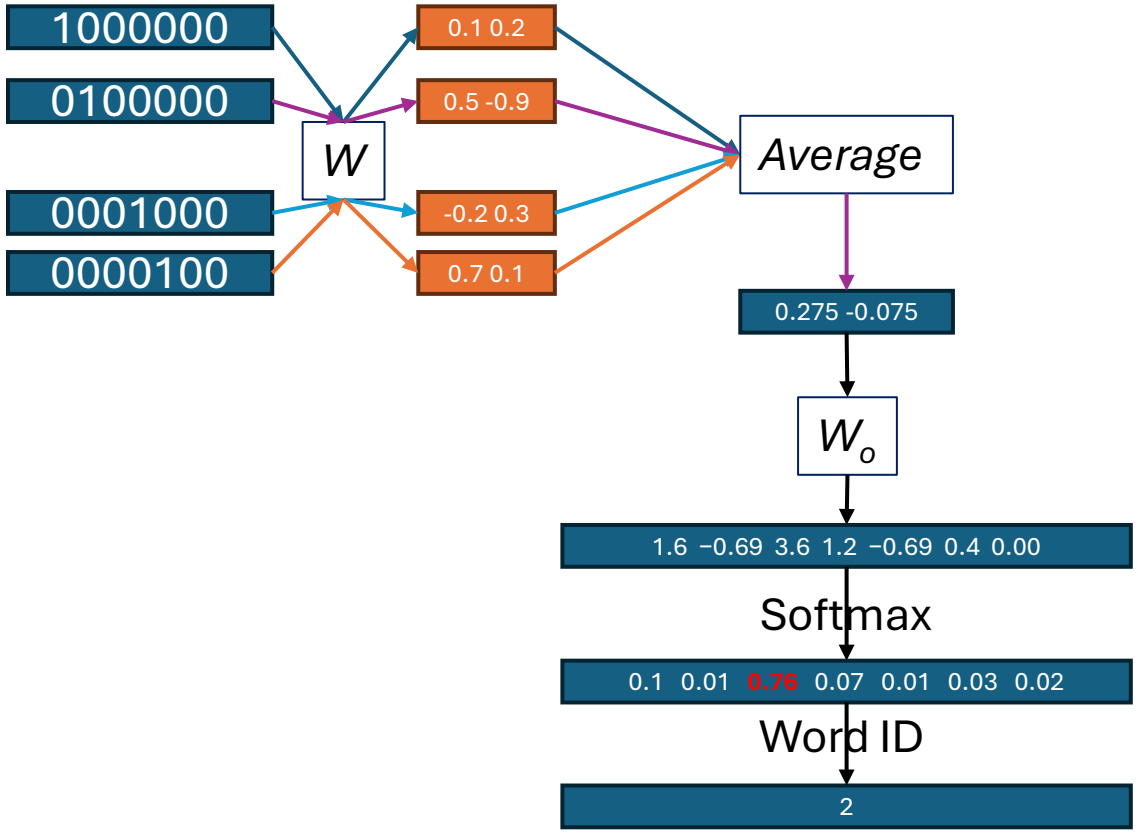
Questions

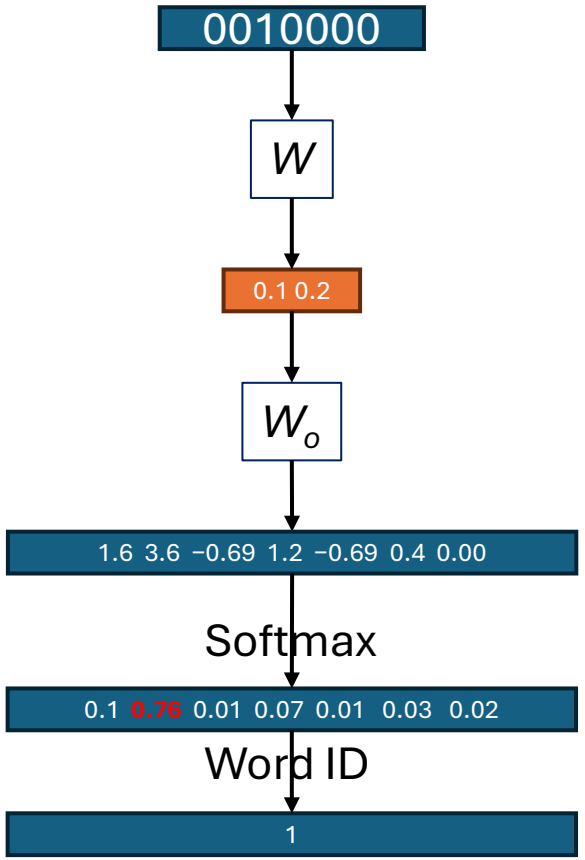
- What problems does a feedforward network encounter when processing sentences?
- What is a timestep in an RNN?
- In RNN computation, what does the hidden state h_t at each time step depend on?

Word Embedding

- Why Do We Need Word Embeddings?
  - Efficient Representation
    - Dense vectors (e.g., 100-300 dimensions) reduce memory usage
  - Capture Semantic Relationships
    - One-hot Embeddings are just random IDs
    - For example: happy/glad
    - Similar words have similar vector representations
      - Cosine Similarity

CBOW and Skip-Gram





Questions

- What are the limitations of one-hot embedding?
- CBOW and Skip-Gram
  - Which model predicts the target word according to the context?
  - Which part of the diagram on the last page represents the word embeddings?
- What is the softmax function?

Byte Pair Encoding (BPE)

Algorithm 1 Learn BPE operations

```
import re, collections

def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
    for i in range(len(symbols)-1):
      pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
         'n e w e s t </w>':6, 'w i d e s t </w>':3}
num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
  print(best)
```

1. Split word into characters → Vocabulary
   - `</w>`
2. Count the frequency of each bigram
   - a b; b c
   - ab c
3. Merge the most frequently appeared bigram
4. Repeat 2 and 3 until some threshold is met
   - Number of merges
   - Vocabulary size

- bpe.py

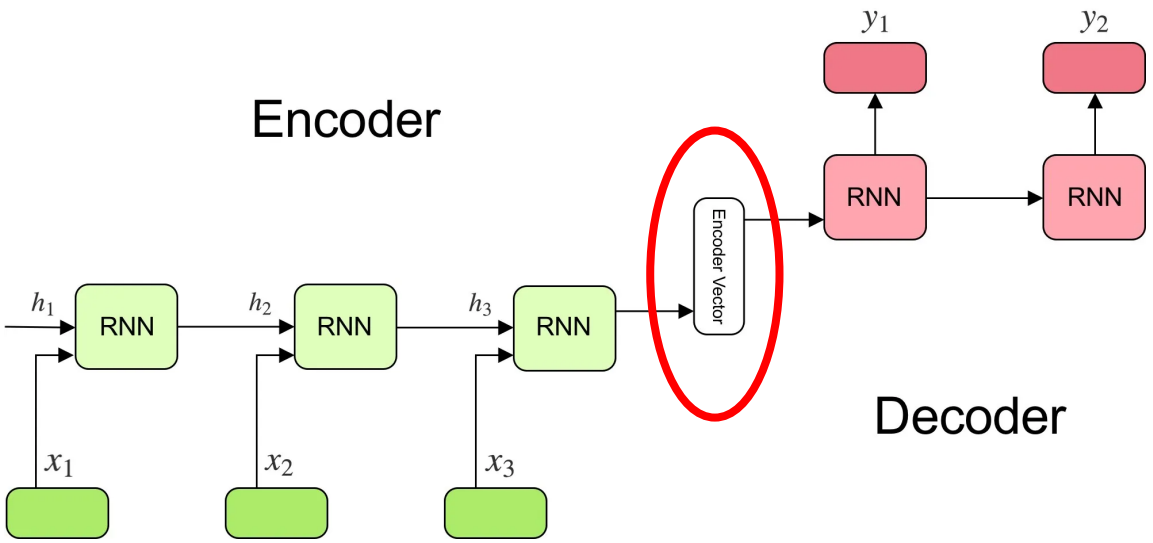
Sennrich R. Neural machine translation of rare words with subword units[J]. arXiv preprint arXiv:1508.07909, 2015.
https://www.pure.ed.ac.uk/ws/portalfiles/portal/174256085/Neural_Machine_Translation_SENNRICH_DOA24052016_VOR_CC_BY.pdf

Questions

- What is the output of BPE?
- What does "pair" mean in BPE?
- When will the BPE algorithm stop?
- How do you tokenize a new word using BPE?

RNN based Encoder-Decoder

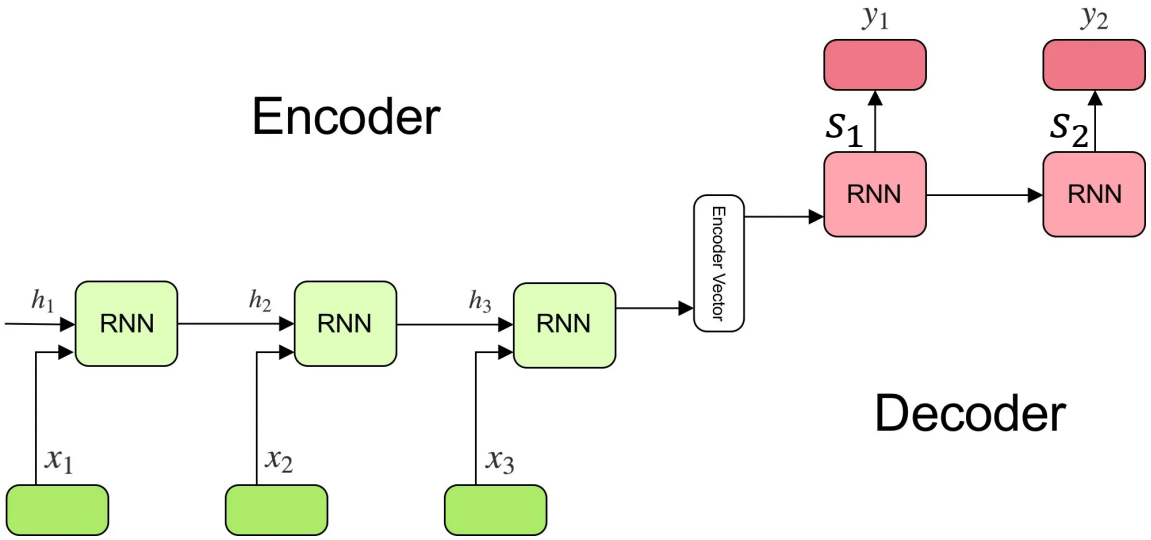
- Encoder
  - The **final hidden state** of the model represents the context/summary of the whole input sequence
  - It acts as the initial hidden state of the decoder part of the model



https://medium.com/analytics-vidhya/encoders-decoders-sequence-to-sequence-architecture-5644efbb3392

RNN based Encoder-Decoder

- Decoder
  - The Decoder generates the output sequence by predicting the next output y_i given the hidden state s_i.
    - $h_t = f(W_h h_{t-1} + W_x x_t + b)$
    - $y_t = g(W_y h_t + b_y)$
  - Input to the Decoder
    - $x_t \rightarrow$ predicted y_{t-1}
      - Starting output y_0: <BOS>
    - Initial hidden state $s_0 = c$
      - The final hidden state of encoder c
    - $s_i = g(W_s s_{i-1} + W_y y_{i-1} + W_c c + b1)$
    - $y_i = softmax(W_o s_i + b2)$



https://medium.com/analytics-vidhya/encoders-decoders-sequence-to-sequence-architecture-5644efbb3392

Questions

- What are the limitations of the RNN-based encoder-decoder architecture?
  - All the context vectors c are the same.
- How is the initial hidden state of the decoder determined?

Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- mapping a query and a set of key-value pairs to an output
- output is computed as a weighted sum of the values
- where the **weight** is computed by a **compatibility function of the query with the corresponding key**.

Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017. https://arxiv.org/pdf/1706.03762

Questions

- How is the dot product calculated?

$$dot\ product(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \cdots + v_N w_N$$

- What is the output of a vector's attention: a vector, a number, or a matrix?
- If matrix X has dimensions $N \times N$ and matrix W has dimensions $N \times D$, what are the dimensions of their product $X \cdot W$? What is the computational complexity of this matrix multiplication? (This is not an exam question.)
  - The output matrix has dimensions $N \times D$.
  - The computational complexity is $O(N^2 D)$

Applications of Attention in Transformer

- Encoder-decoder attention
  - The **queries** come from the previous **decoder** layer
  - The memory **keys** and **values** come from the output of the **encoder**
  - This allows every position in the decoder to attend **over all positions** in the input sequence

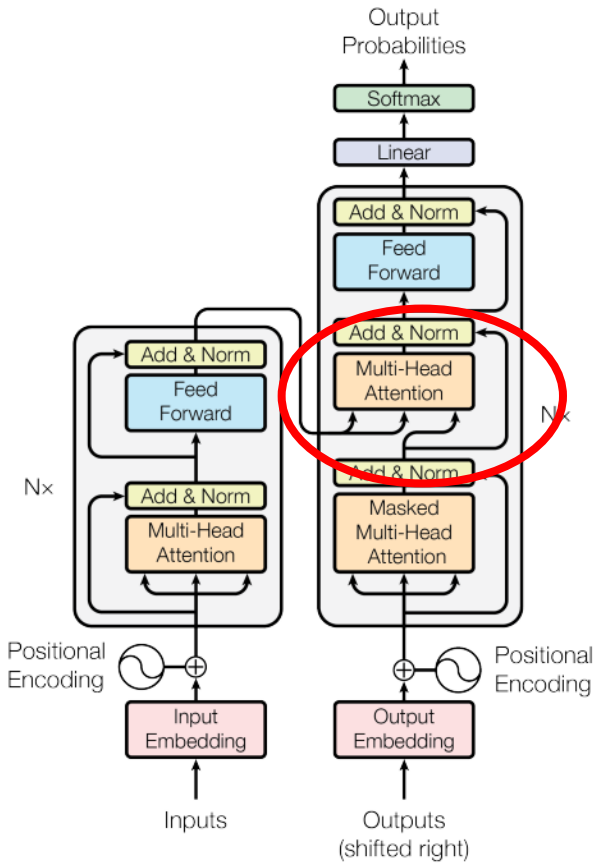


Figure 1: The Transformer - model architecture.

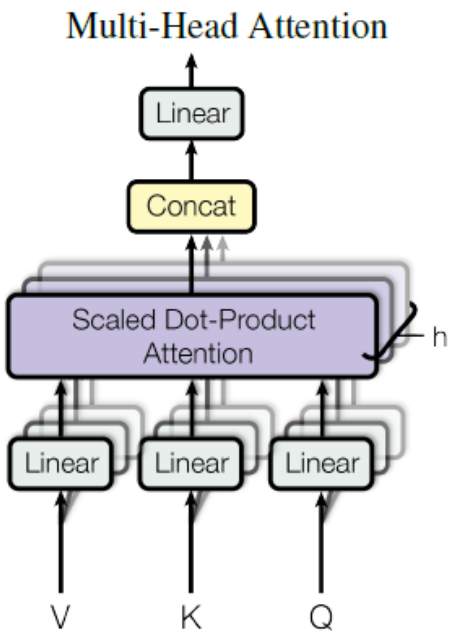
Questions

- What is self-attention?
- What is the difference between self-attention and encoder-decoder attention (cross attention)?
  - The queries are different

Multi Head Attention

- It's **beneficial** to linearly project the queries, keys and values $\boldsymbol{h}$ **times** with **different**, learned linear projections to d_k, d_k, and d_v dimensions, respectively.
  - Before: Map to 512 dimension
  - After: Map to 64 dimension for 8 times
  - In my understanding, we tell the model these 64 dimensions learn different features

- Head number $h = 8, d_k = d_v = \frac{d_{model}}{h} = 64$
- These are concatenated and once again projected, resulting in the final values.



$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_\mathrm{h})W^O$$
$$\text{where } \mathrm{head}_\mathrm{i} = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Questions

- What is the purpose of multi-head attention in Transformers?
- Why do we use different learned projections for Query, Key, and Value in multi-head attention?
  - The purpose is to capture more diverse contextual information.

Embeddings and Softmax

- We use **learned embeddings** to convert the input tokens and output tokens to vectors of dimension d_{model}.
- We also use the usual **learned linear transformation and softmax function** to convert the decoder output to predicted next-token probabilities.
- In our model, we **share the same weight matrix** between the two embedding layers and the pre-softmax linear transformation
- In the embedding layers, we multiply those weights by $\sqrt{d_{model}}$
  - Numerical Stability

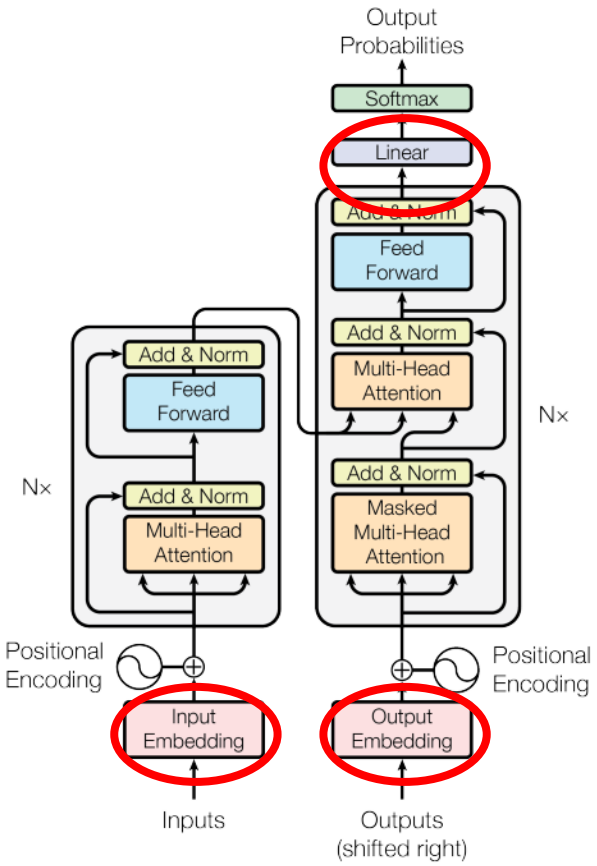


Figure 1: The Transformer - model architecture.

Questions

- How many word embedding methods are you familiar with?
  - Word2Vec
  - One-hot
  - Learned embedding
    - Embedding table in PyTorch
- Is BPE a word embedding method?
  - No, BPE is a subword tokenization method, not a word embedding method.

Positional Encoding

- Why do we need positional encoding?
  - To make use of the order of the sequence
  - '*Book a book in the library*'

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension.

- Each position is represented by a vector, not just a single number.
- How do we calculate the value for each dimension in the vector?

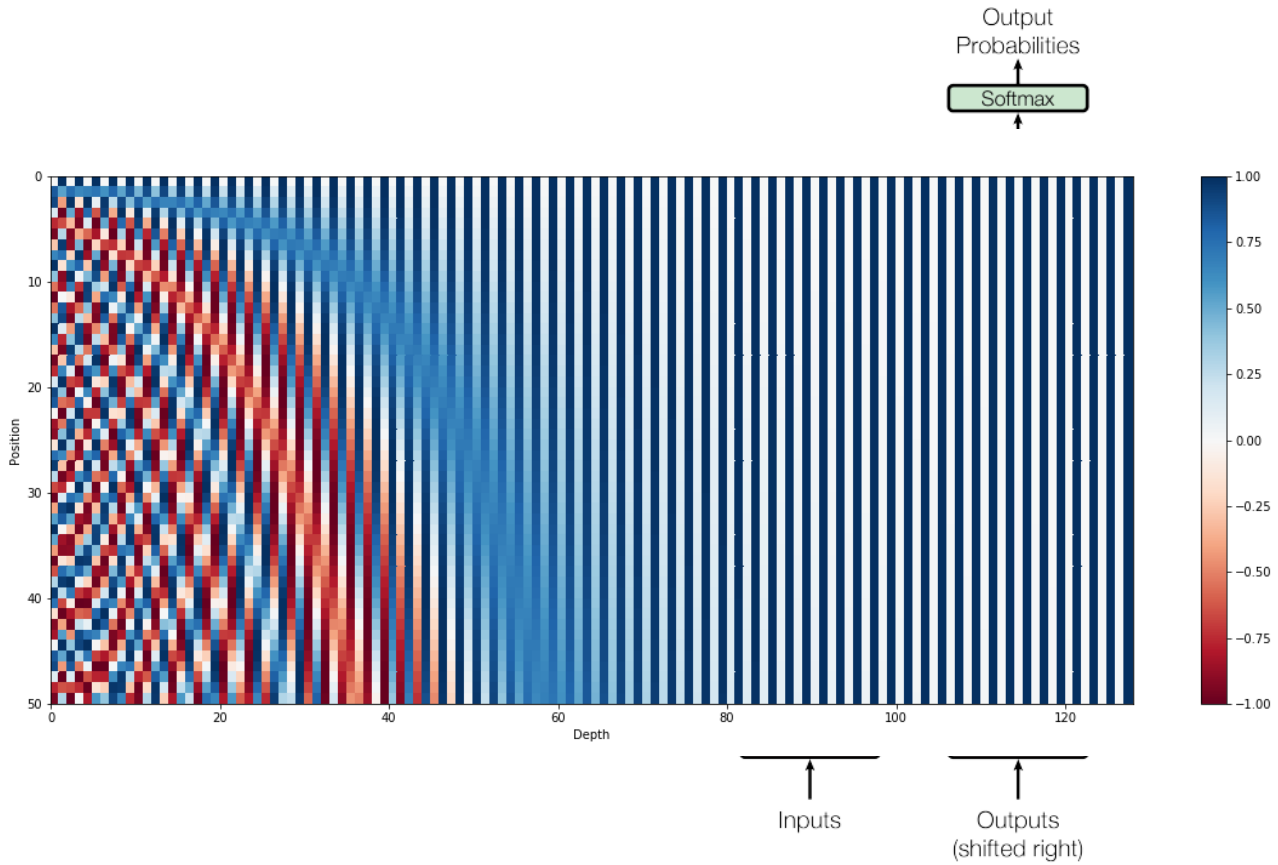


Figure 1: The Transformer - model architecture.

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Questions

- In a Transformer, is a positional embedding a number or a vector?
  - A vector
- How is the value of each dimension in the vector determined?

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

BERT - Input

- A “sequence” refers to the input token sequence to BERT, which may be a single sentence or two sentences packed together.
- We use **WordPiece** embeddings with a 30,000 token vocabulary.
- The first token of every sequence is always a special **classification** token ([CLS]).
  - The final hidden state corresponding to this token is used as the aggregate **sequence representation** for classification tasks.
- Sentence pairs are packed together into a single sequence.
  - <Question, Answer>
  - First, we separate them with a special token ([SEP]).
  - Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B.

Questions

- What is the full name of BERT?
  - Bidirectional Encoder Representations from Transformers
  - Why is BERT bidirectional?
- What are the inputs of BERT?

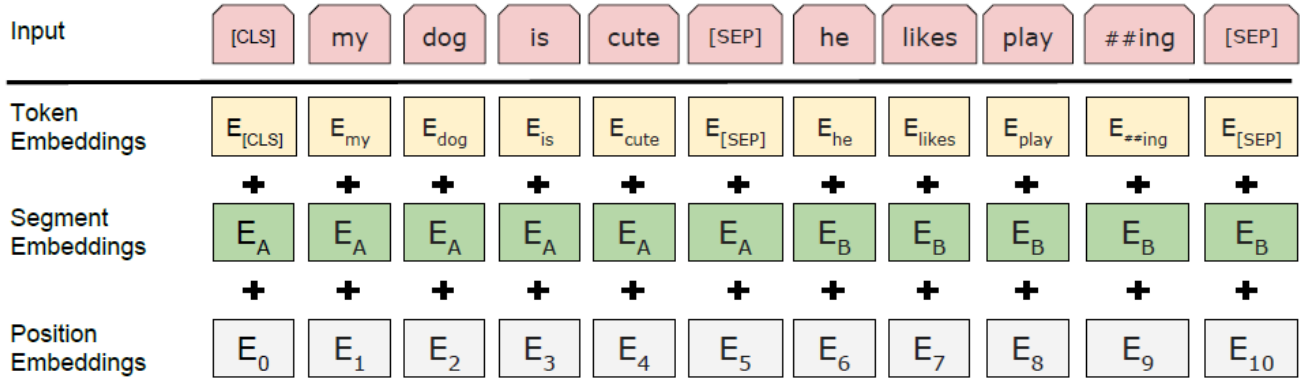


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

BERT - Pre-Training

Input = [CLS] the man went to [MASK] store [SEP]

he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]

penguin [MASK] are flight ##less birds [SEP]

Label = NotNext

$$\text{Loss}_{total} = \text{Loss}_{MLM} + \text{Loss}_{NSP}$$

Applications of Attention in Transformer

- Masked self-attention layers in the decoder
  - We need to prevent leftward information flow in the decoder to preserve the auto-regressive property.
    - For example, if you have only generated $[y_1, y_2]$, the model cannot access or "see" the values of $[y_3, y_4]$
  - Mask out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

$$\mathrm{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

- Formula (with mask M):

$$\mathrm{Attention}(Q, K, V) = \mathrm{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

  - M is the triangular mask ensuring causal behavior.

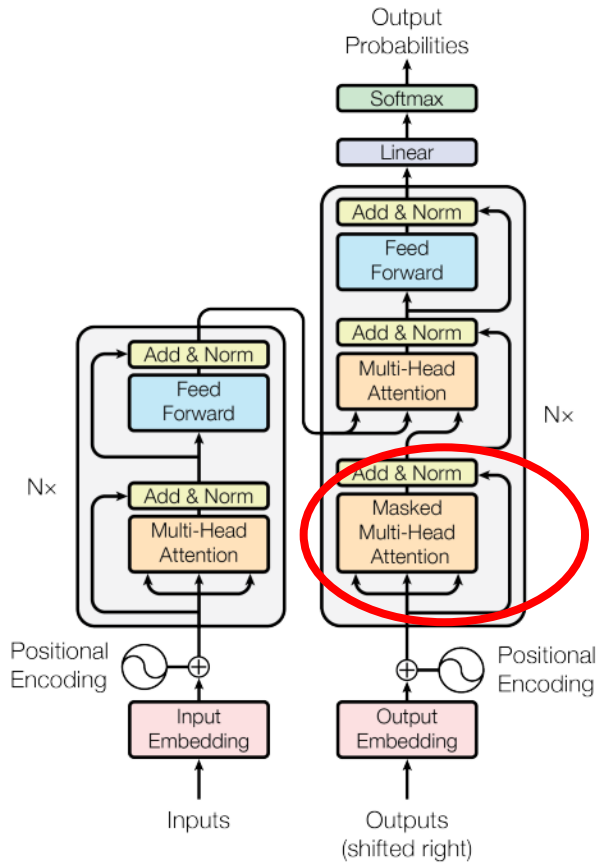


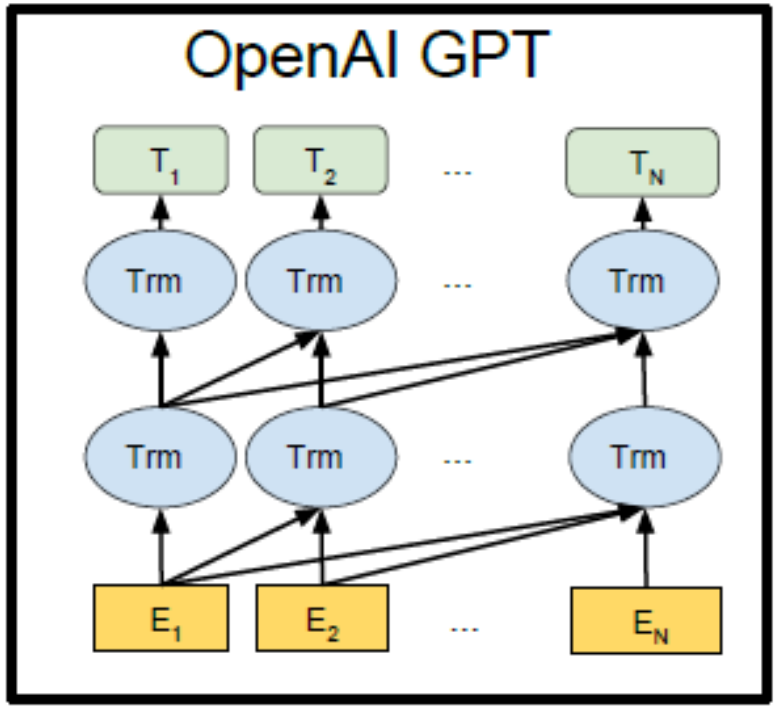
Figure 1: The Transformer - model architecture.

Questions

- What do [CLS] and [SEP] mean?
- What is the Masked Language Model?
- If we input 'book a book' into BERT, will the output (token embedding) of the first 'book' and the second 'book' be the same?
  - No
  - What about Word2Vec?

Decoder-Only Models

- Uses the **Transformer decoder**, which processes input sequentially in a unidirectional way
- left-to-right
  - Causal Language Modeling
  - Predicts the next token given previous ones
- Focus
  - Generating text



Devlin J. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018. https://arxiv.org/pdf/1810.04805

N-Gram Language Models

- A statistical model that predicts the next word based on the previous words.
- Example: In a 3-gram model, "I love ___" might predict "pizza" based on counts from the dataset.

- Fixed-length context, limited scalability.

Bigram $P(w_1, w_2, \dots, w_T) = \prod_{t=1}^{T} P(w_t | w_{t-1})$

Trigram $P(w_1, w_2, \dots, w_T) = \prod_{t=1}^{T} P(w_t | w_{t-1}, w_{t-2})$

- Causal Language Modelling
  - Predicts the next word in a sequence based on all previous words
  - Model long contexts

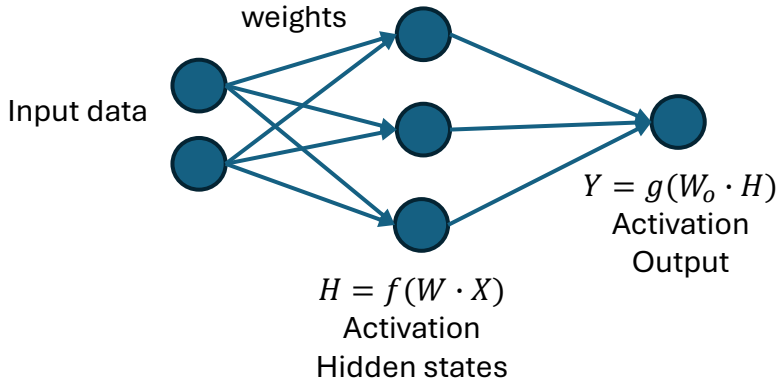
$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^{T} P(w_t | w_{t-1}, w_{t-2}, \dots, w_1)$$

Questions

- What is Causal Language Modelling
- What are unigram, bigram, and trigram?
  - We use these in BLEU

How Parameter Count is Calculated in GPT

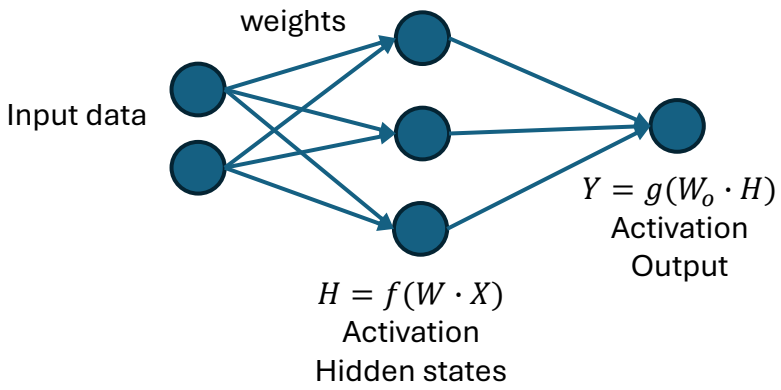
- Feedforward Layers
  - Consist of two linear transformations with an intermediate activation function.
  - Parameters
    - $Feedforward\ Parameters = (d_{model} \times d_{ff}) + (d_{ff} \times d_{model})$
    - where d_{ff} is the hidden size of the feedforward network (typically $4 \times\ d_{model}$)
    - $d_{model}\ =\ 12288\ d_{ff}\ = 49152$
    - $Feedforward\ Parameters = 1207M$



$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Questions

- Ignore the bias, how many parameters are there in this network?
  - 2*3+3*1=9
  - How many W?
    - 2
  - Dimensions of the W?
    - 2*3 and 3*1



LLM - Quantization

- Convert floating-point numbers (like FP32) to lower-precision integers (like INT8)
- To reduce the model size and speed up inference

- Example: Converting Float to Integer
  - Assume the floating-point range is [−1.0,1.0], The integer range is [0,255]
  - Calculate **scale** and **zero point**
  - Scale determines how floating-point values are scaled to integers.

$$\text{scale} = \frac{\text{float_max} - \text{float_min}}{\text{int_max} - \text{int_min}}$$

$$scale = \frac{1-(-1)}{255-0} = \frac{2}{255} \approx 0.00784$$

  - Zero Point: This aligns the ranges.

$$z_p = Q_{min} - \frac{F_{min}}{scale}$$

$$z_p = 0 - \frac{-1}{0.00784} \approx 128$$

LLM - Quantization

$$scale = \frac{1-(-1)}{255-0} = \frac{2}{255} \approx 0.00784$$

$$z_p = 0 - \frac{-1}{0.00784} \approx 128$$

- Quantization formula

$$Q_{min} = round(\frac{F_{float}}{scale}) + z_p$$

- F_{float} =-1
- $Q_{min} = round\left(\frac{-1}{0.00784}\right) + 128 = round(-127.55) + 128 = 0$

Questions

- Why do we use quantization in inference?
  - To reduce the model size and speed up inference

Perplexity

$$perplexity(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1}, \dots, w_1)}}$$

- The higher the probability, the lower the perplexity.
- **Lower perplexity means the model is more confident in its predictions**, while higher perplexity suggests it is more uncertain.
- If a model is perfectly predicting the test data (assigns probability 1 to the correct words), its perplexity is 1.

Questions

- If a model's perplexity on a test set is 1.92, does it indicate good performance?
  - Yes

Question Answering - TriviaQA

- TriviaQA consists of trivia-style questions, where models must find and **extract** the correct answer from a collection of supporting documents.
- TriviaQA includes **95K question-answer pairs** and independently gathered evidence documents, six per question on average.
- **F1 is computed based on word overlap** between the predicted answer and the gold (correct) answer.

Formula for F1 Score

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- **Precision** = (Number of overlapping words) / (Total words in the predicted answer)
- **Recall** = (Number of overlapping words) / (Total words in the ground-truth answer)

https://nlp.cs.washington.edu/triviaqa/

Questions

- Precision = 40/(40+20) = 0.67
- Recall = 40/(40+10) = 0.8
- F1 = 2*0.67*0.8/(0.67+0.8) = 0.73

	Positive (Predicted = 1)	Negative (Predicted = 0)
Positive (Actual = 1)	40	10
Negative (Actual = 0)	20	30

Translation - BLEU

- **Bilingual Evaluation Understudy**

$$BLEU = BP \times \exp(\sum_{i=1}^{N} w_n \log p_n)$$

- Where:
  - p_n is the precision of n-grams (unigram, bigram, etc.)
  - Number of matched n-grams divided by number of total n-grams in candidate
  - Any $p_n = 0, \log 0 = -\infty, BLEU = e^{-\infty} = 0$
  - w_n is the weight assigned to each n-gram level (typically equal weights)
  - BP: **brevity penalty**, which prevents shorter sentences from getting artificially high scores
  - $BP = \begin{cases} 1 & if\ c > r \\ e^{(1-r/c)} & if\ c \leq r \end{cases}$ where c is the length of generated text, and r is the length of reference text
  - N is set to 4 or 5

BERTScore

- A good translation may use alternate words or paraphrases
- Instead of exact matches, we compute **token similarity** using contextual embeddings

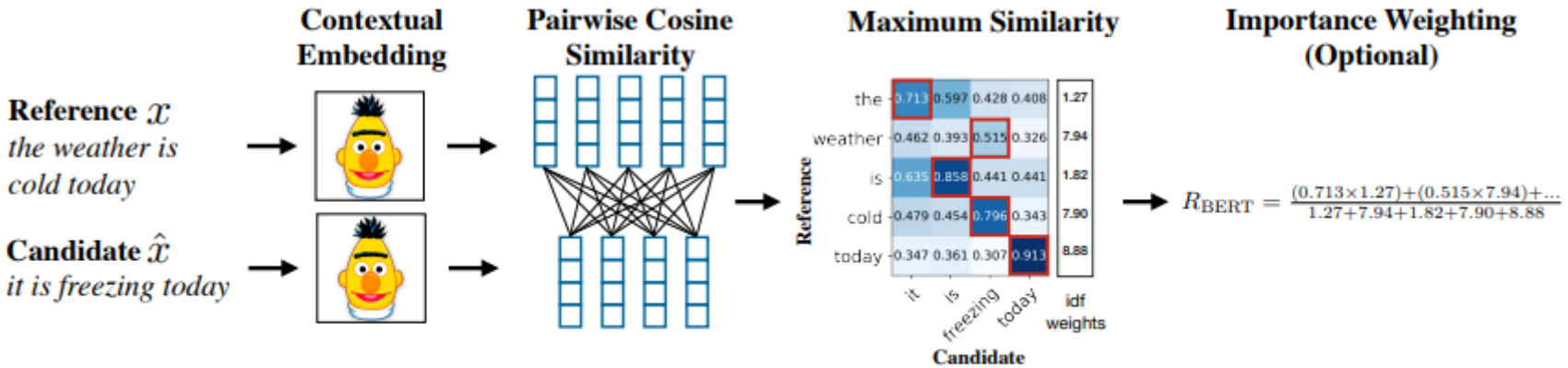


Figure 1: Illustration of the computation of the recall metric R_{BERT}. Given the reference x and candidate $\hat{x}$, we compute BERT embeddings and pairwise cosine similarity. We highlight the greedy matching in red, and include the optional idf importance weighting.

Zhang T, Kishore V, Wu F, et al. Bertscore: Evaluating text generation with bert[J]. arXiv preprint arXiv:1904.09675, 2019. https://arxiv.org/pdf/1904.09675

BERTScore

```
import evaluate
predictions = ["the weather is cold"]
references = [
     ["it is freezing"]
]
bleu = evaluate.load("bleu")
results = bleu.compute(predictions=predictions, references=references)
print(results)
bertscore = evaluate.load("bertscore")
results = bertscore.compute(predictions=predictions, references=references[0], lang="en")
print(results)
```

{**'bleu': 0.0,** 'precisions': [0.25, 0.0, 0.0, 0.0], 'brevity_penalty': 1.0, 'length_ratio': 1.3333333333333333, 'translation_length': 4, 'reference_length': 3}

{'precision': [0.9036330580711365], 'recall': [0.9348080158233643], **'f1': [0.9189561605453491]**, 'hashcode': 'roberta-large_L17_no-idf_version=0.3.12(hug_trans=4.41.2)'}

Questions

- Given $BLEU = BP \times \exp\left(\sum_{i=1}^{N} w_n \log p_n\right)$, where N=4, BP=1
  - If there is no BP, which type of sentence is more likely to get a higher score, long or short?
- What are the limitations of BLEU?
- What is BERTScore?
- What is Cosine Similarity

Summary

- Attention is all you need
  - Input: token & embedding
  - Model structure
  - Output: BERT
- Evaluation
  - F1, BertScore, BLEU, Perplexity